

Problem A. Live Love

The maximum possible number of consecutive PERFECT strings is clearly m : put all m copies of PERFECT together.

For the minimum possible maximum run length, binary search the answer x . If no more than x consecutive PERFECT strings are allowed, then between every two full groups of x PERFECT strings we must insert at least one NON-PERFECT. Thus the shortest total length needed for this bound is

$$\text{need}(x) = \begin{cases} \frac{m}{x}(x+1) - 1, & x \mid m, \\ \left\lfloor \frac{m}{x} \right\rfloor (x+1) + (m \bmod x), & x \nmid m. \end{cases}$$

The value x is feasible exactly when $\text{need}(x) \leq n$. This condition is monotone, so binary search finds the minimum x . If $m = 0$, both answers are 0.

The time complexity is $\mathcal{O}(\log m)$.

Problem B. Red Black Tree

Let $\text{cost}(v_i)$ be the cost of vertex v_i before inserting the new red vertex. For a query $v_1, v_2, \dots, v_k$, sort the queried vertices in non-increasing order of their current cost. If no new red vertex is used, the maximum cost is $\text{cost}(v_1)$.

To make the maximum cost smaller than $\text{cost}(v_i)$, the inserted red vertex must change the costs of all vertices $v_1, v_2, \dots, v_i$. The best place to put it is their lowest common ancestor. If $d(v)$ denotes the distance from the root to v , then the resulting maximum cost after covering the first i vertices is

$$\max \left(\max_{1 \leq j \leq i} d(v_j) - d(\text{lca}(v_1, v_2, \dots, v_i)), \text{cost}(v_{i+1}) \right),$$

where $\text{cost}(v_{k+1})$ is treated as 0. Enumerate $i = 1, 2, \dots, k$ and take the minimum.

If the path from the chosen LCA to some v_j already contains another red vertex, the value computed above is at least $\text{cost}(v_j)$ and cannot improve over the case $i = j - 1$. Thus this issue does not affect the minimum.

The time complexity is $\mathcal{O}(n \log n + \sum k_i \log \sum k_i)$.

Problem C. Halting Problem

The whole state of the computer consists of only two values: the line currently being executed and the value of the register r .

Run BFS over these states. If a state with line $n + 1$ is reachable, the program can halt; otherwise it cannot. Since r is a p -bit value and here $p = 8$, there are only $n \cdot 2^p$ possible states.

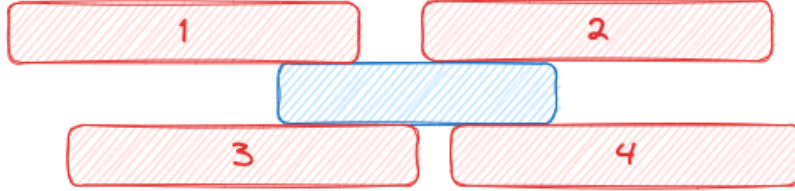
The time complexity is $\mathcal{O}(n \cdot 2^p)$.

Problem D. Pixel Art

Treat each black horizontal or vertical segment as a rectangle. Sweep rows from top to bottom, maintaining in a map which column intervals of the current row are covered and which connected component each interval belongs to.

When a new rectangle appears, insert it into the map and check whether it touches another rectangle in the upward, left, or right direction. Whenever two rectangles touch, merge their components with Union-Find.

At first this gives $\mathcal{O}(n + t \log k)$, where t is the number of adjacent rectangle pairs. In fact, $t = \mathcal{O}(k)$. For vertical adjacency, charge every adjacent pair to the shorter rectangle. A shorter interval can touch at most four longer intervals, as illustrated below; horizontal adjacency is symmetric.



Therefore the total number of relevant adjacency pairs is linear in k , and the final complexity is $\mathcal{O}(n + k \log k)$.

Problem E. Infinite Parenthesis Sequence

Let $f(k, i)$ be the position of the i -th left parenthesis in B^k ; i may be zero or negative. Let $g(k, p)$ be the largest i satisfying $f(k, i) \leq p$. Then the number of left parentheses in interval $[l, r]$ is

$$g(k, r) - g(k, l - 1).$$

The value $g(k, p)$ can be obtained by binary searching i . Since only differences of g are used, the exact choice of the zeroth left parenthesis is arbitrary. We choose the first left parenthesis in the original finite string.

For the transition of $f(k, i)$:

- If the character after $f(k, i)$ is a right parenthesis, then $f(k + 1, i) = f(k, i) + 1$.
- If the character after $f(k, i)$ is a left parenthesis, then $f(k + 1, i) = f(k, i + 1) - 1$.

Thus

$$f(k + 1, i) = \min(f(k, i) + 1, f(k, i + 1) - 1).$$

Suppose $f(k, i)$ is obtained from $f(0, j)$. While the first coordinate goes from 0 to k , the second term of the minimum is chosen exactly $j - i$ times, and the first term is chosen exactly $k - (j - i)$ times. Hence

$$f(k, i) = \min_{i \leq j \leq i+k} (f(0, j) + k - 2(j - i)).$$

The decision interval has length k . Let m be the number of left parentheses in the original finite string and define $F(j) = f(0, j) - 2j$. Since $f(0, j + m) - f(0, j) = n$,

$$F(j + m) = F(j) + (n - 2m).$$

For $k \leq m$, preprocess $F(0), F(1), \dots, F(2m - 1)$ with RMQ, then map $[i, i + k]$ to $[i \bmod m, i \bmod m + k]$.

For larger k , the interval can still be shortened to length at most m :

- If $n - 2m \geq 0$, the minimum appears in $i \leq j < i + m$.
- If $n - 2m < 0$, the minimum appears in $i + k - m < j \leq i + k$.

Then the same RMQ preprocessing can be reused. For the binary search on g , the range $[-3 \times 10^9, 3 \times 10^9]$ is sufficient.

The time complexity is $\mathcal{O}(n \log n + q \log(n + k - l + r))$.

Problem F. Chaleur

The input graph is guaranteed to be splittable into two parts A and B , where A is a clique and B is an independent set. First find a maximum possible clique A .

If the graph contains a clique of size s , then the s -th largest degree is at least $s - 1$. This determines the largest possible value of $|A|$, namely the maximum clique size. The vertices with the largest $|A|$ degrees form A ; by the guarantee, the remaining vertices have no edges among themselves and form B .

Number of maximum cliques

Vertices in B can only be adjacent to vertices in A , so their degree is at most $|A|$. If some vertex in B had degree $|A|$, it could be added to A to make a larger clique, a contradiction. Therefore the maximum degree among vertices in B is $|A| - 1$.

If a vertex $u \in B$ has degree $|A| - 1$, and v is the unique vertex of A not adjacent to u , then $(A - \{v\}) \cup \{u\}$ is also a maximum clique. Therefore the number of maximum cliques is

$$1 + \#\{u \in B : \deg(u) = |A| - 1\}.$$

Number of maximum independent sets

Since vertices in A are pairwise adjacent, at most one vertex from A can be used together with vertices from B .

- If some $u \in A$ has degree $|A| - 1$, then u has no neighbors in B , so $B \cup \{u\}$ is a maximum independent set. The number of such sets is the number of vertices in A with degree $|A| - 1$.
- Otherwise, if some $u \in A$ has degree $|A|$, then u is adjacent to exactly one vertex $v \in B$, so $(B - \{v\}) \cup \{u\}$ is also a maximum independent set. Including B itself, the number of maximum independent sets is one plus the number of vertices in A with degree $|A|$.
- Otherwise, every vertex in A has degree greater than $|A|$, meaning that it is adjacent to at least two vertices in B . Then no replacement improves on B , and the number of maximum independent sets is 1.

The dominant cost is sorting by degree. The time complexity is $\mathcal{O}(n \log n + m)$.

Problem G. Couleur

Deleted positions split the sequence $a_1, a_2, \dots, a_n$ into several segments. The number of inversions in a subarray never exceeds the number of inversions in the whole segment containing it, so it is enough to maintain the inversion count of every current segment and take the maximum.

Use a map to maintain the segments and their inversion counts. When deleting position x from a segment $[l, r]$, split it into $[l, x - 1]$ and $[x + 1, r]$. Let $\text{rev}(l, r)$ be the inversion count of $[l, r]$.

If $x - l < r - x$, compute the smaller left segment directly and derive the right segment by subtraction:

$$\begin{aligned} \text{rev}(x + 1, r) &= \text{rev}(l, r) - \text{rev}(l, x - 1) \\ &\quad - \#\{\text{inversions with one element in } [l, x - 1] \text{ and the other in } [x + 1, r]\} \\ &\quad - \#\{\text{inversions involving } a_x\}. \end{aligned}$$

The case $x - l > r - x$ is symmetric.

Use a persistent segment tree over the value domain. It answers queries of the form

$$f(i, u, v) = \#\{1 \leq j \leq i : u \leq a_j \leq v\},$$

which lets us compute the subtracted terms in $\mathcal{O}(\min(x-l, r-x) \log n)$ time.

By heuristic splitting, every time an element is scanned directly, the segment containing it has at most half its previous length. Hence each element participates in direct computation at most $\mathcal{O}(\log n)$ times, and the total time complexity is $\mathcal{O}(n \log^2 n)$.

Problem H. Traveling on the Axis

Consider the relation between $t(p, q-1)$ and $t(p, q)$. It is easy to see that

$$t(p, q) = \begin{cases} t(p, q-1) + 2, & s_{q-1} = s_q, \\ t(p, q-1) + 1, & s_{q-1} \neq s_q. \end{cases}$$

Let

$$f(q) = \sum_{p=0}^{q-1} t(p, q).$$

Then

$$f(q) = \begin{cases} f(q-1) + 2(q-1) + t(q-1, q), & s_{q-1} = s_q, \\ f(q-1) + (q-1) + t(q-1, q), & s_{q-1} \neq s_q. \end{cases}$$

With the initial value $f(0) = 0$, the required answer is

$$\sum_{q=1}^n f(q).$$

This recurrence can be evaluated in a single scan. The time complexity is $\mathcal{O}(n)$.

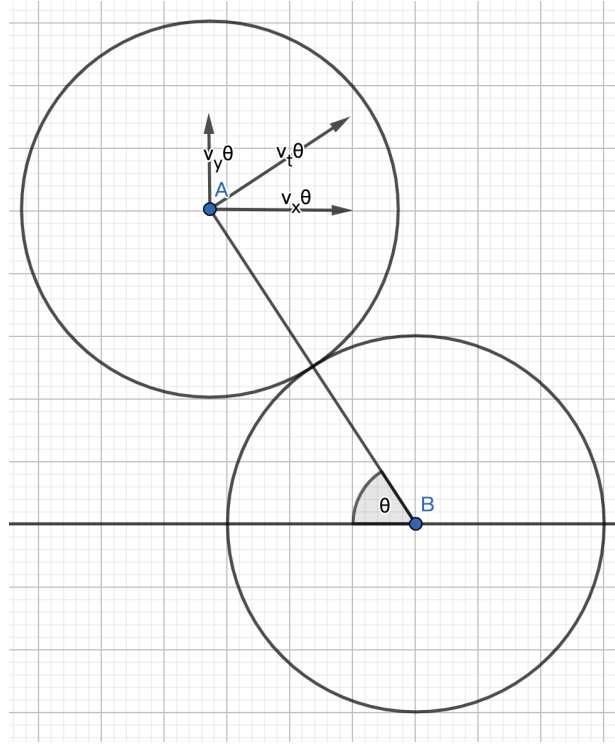
Problem I. Kuririn MIRACLE

Let the center of the first car be A and the center of the obstacle car be B . A simple upper bound is obtained by letting circle A follow circle B along the x -axis all the way to the destination; this takes time d/v .

Since circle A is faster than circle B , we may instead keep them tangent, make A go around the upper side of B until some angle, and then move A straight to the destination. We compute these two parts separately.

First part of the path

The first part of the motion of A can be decomposed into a uniform translation following B at speed v , and a circular motion around B with radius $2r$ and variable tangential speed.



Let θ be the angle between AB and the negative direction of the x -axis. Let $v_t(\theta)$ be the tangential speed of A relative to B , and let $v_x(\theta), v_y(\theta)$ be its components in the positive x and positive y directions. Then

$$\begin{cases} v_x(\theta) = v_t(\theta) \sin \theta, \\ v_y(\theta) = v_t(\theta) \cos \theta, \\ (v_x(\theta) + v)^2 + (v_y(\theta))^2 = (2v)^2. \end{cases}$$

Solving gives

$$v_t(\theta) = v \left(\sqrt{\sin^2 \theta + 3} - \sin \theta \right).$$

If the first part continues until $\theta = \alpha$, its duration is

$$t_1(\alpha) = \int_0^\alpha \frac{2r \, d\theta}{v_t(\theta)}.$$

This integral has no convenient closed form, so it can be computed numerically, for example with adaptive Simpson integration.

Second part of the path

After the first part, the coordinates of A are

$$\begin{cases} x = vt_1(\alpha) + 2r - 2r \cos \alpha, \\ y = 2r \sin \alpha. \end{cases}$$

Then A moves from this point to the destination in a straight line, so the second duration is

$$\frac{\sqrt{(x-d)^2 + y^2}}{2v}.$$

It remains to choose α . Let β be the angle between the line from A to the destination and the negative direction of the x -axis just before the second part begins. In the coordinate system centered at B , the point A starts from

$$\vec{a} = (x - (2r + vt_1(\alpha)), y)$$

and moves in the direction

$$\vec{w} = (2v \cos \beta - v, -2v \sin \beta).$$

To ensure that the distance between A and B is not decreasing at the switch point, the angle between $\vec{a}$ and $\vec{w}$ must be at most 90° , i.e.

$$\vec{a} \cdot \vec{w} \geq 0.$$

Binary search α and check this condition.

The complexity is the cost of the binary search multiplied by the cost of the numerical integration.

Problem J. Press the Button

Let $L = \text{lcm}(a, c)$. The pattern of button presses repeats every L seconds. During one period, BaoBao presses the button $c/\text{gcd}(a, c)$ times, each time pressing it b times, while DreamGrid presses the button $a/\text{gcd}(a, c)$ times, each time pressing it d times. Therefore direct simulation over one period costs $\mathcal{O}(a+c)$.

- If $t < L$, simply simulate all button presses up to time t .
- If $t \geq L$, first simulate the interval $[0, L - 1]$. Let x be the increase of the counter in this interval, and let $w = 1$ if the LED is still on at the beginning of the next period, otherwise $w = 0$. Next simulate only the final $t \bmod L$ seconds and let the resulting counter increase be y .

Then the answer is

$$(x + w) \left\lfloor \frac{t}{L} \right\rfloor + y.$$

The time complexity remains $\mathcal{O}(a + c)$.

Problem K. XOR Clique

The key observation is that all numbers in a valid set S must have the same highest set bit.

Indeed, suppose S contains two numbers x and y such that the highest set bit of x is 2^p , while the highest set bit of y is $2^q < 2^p$. Then

$$x \oplus y \geq 2^p > y,$$

which violates the required condition.

Conversely, if two numbers both have highest set bit 2^p , then the p -th bit of their xor is 0, so their xor is strictly smaller than the smaller of the two numbers.

Therefore, count how many input numbers have each possible highest set bit, and choose the largest group as the answer. The time complexity is $\mathcal{O}(n \log \max_i a_i)$.